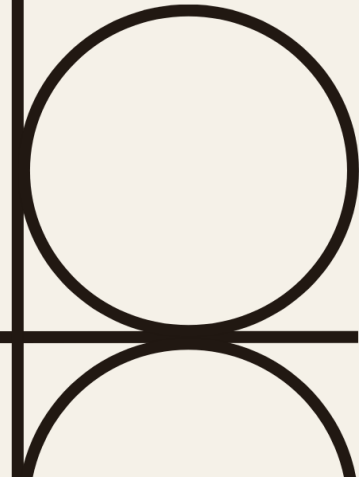




GESTOR DE NOTAS

byron alberto iquic sequen

Este sistema permite registrar, consultar, editar, eliminar y organizar notas de cursos académicos, junto con funcionalidades de búsqueda, ordenamiento y revisión. Está diseñado para ejecutarse en ordenador, con una estructura modular y clara, ideal para estudiantes o docentes que deseen llevar control de sus evaluaciones.

1. Registro de Cursos

- Solicita nombre y nota del curso.
- Valida campos vacíos y rango de nota.
- Almacena en historialcursos.

2. Visualización de Cursos

- Muestra todos los cursos registrados con sus notas.

3. Promedio General

- Calcula el promedio de todas las notas registradas.

4. Conteo de Aprobados/Reprobados

- Clasifica cursos según si la nota es ≥ 60 .

5. Búsqueda Lineal

- Recorre el diccionario para encontrar un curso por nombre.
- Registra la cantidad de veces que se ha buscado.

6. Edición de Curso

- Permite cambiar el nombre o la nota de un curso existente.

- Registra los cambios en el historial.

7. Eliminación de Curso

- Elimina un curso tras confirmación del usuario.
- Registra el evento en el historial.

8. Ordenamiento por Nota

- Aplica ordenamiento burbuja para mostrar cursos de mayor a menor nota.

9. Ordenamiento Alfabético

- Aplica ordenamiento por inserción para mostrar cursos de A a Z.

10. Búsqueda Binaria

- Ordena los nombres de cursos y aplica búsqueda binaria por nombre.

11. Simulación de Cola de Revisión

- Recorre los cursos en orden de ingreso simulando una cola FIFO.

12. Historial de Acciones

- Muestra todas las acciones registradas en el sistema.

13. salir

finaliza la ejecución del programa.

programa Utilizada

Lenguaje de programación: Python

Algoritmos aplicados: Búsqueda lineal, búsqueda binaria, ordenamiento burbuja, ordenamiento por inserción

INIALIZACIÓN Y MENÚ

```
print("bienvenido al sistema de registro de notas")
nom_pers = input("Ingrese su nombre para empezar: ")
if nom_pers == "":
    print("El nombre no puede estar vacío. Reinicie el programa.")
    exit()
print("Nombre registrado, ¡HERE WE GO!")
while True:
```

```

print("menu principal: ")
print("1. Agregar curso")
print("2. Mostrar cursos")
print("3. Calcular promedio")
print("4. Conteo de cursos aprobados y reprobados")
print("5. Buscar curso por nombre (lineal)")
print("6. Actualizar o editar nombre del curso/nota")
print("7. Eliminar curso")
print("8. Ordenar por nota")
print("9. Ordenar por nombre")
print("10. Buscar curso por nombre (binaria)")
print("11. Simular cola de revisión")
print("12. Mostrar historial de cambios")
print("13. Salir")
try:
    opcion = int(input("Ingrese el número de la opción: "))
except ValueError:
    print("Por favor, ingrese un número válido.")
    continue
if opcion == 1:
    registrarnotacurso()
elif opcion == 2:
    mostrarcursos(historialcursos)
elif opcion == 3:
    promediogeneral(historialcursos)
elif opcion == 4:
    reproapro(historialcursos)
elif opcion == 5:
    busquedalineal_func(historialcursos, busquedalineal, historialgeneral)
elif opcion == 6:
    editarcursonota(historialcursos, historialgeneral)
elif opcion == 7:
    eliminarcurso(historialcursos, historialgeneral)
elif opcion == 8:
    def ordenar_por_nota(cursos, historial):
elif opcion == 9:
    def ordenar_alfabetico(cursos, historial):
elif opcion == 10:
    busqueda_binaria(cursos=historialcursos, historial=historialgeneral)
elif opcion == 11:
    simular_cola_revision(historialcursos)
elif opcion == 12:
    mostrarhistorial(historialgeneral)
elif opcion == 13:
    print(f'Gracias por usar el sistema, {nom_pers}, adiosito')
    break
else:
    print("ingrese una opcion valida")

```

registro de notas

```

def registrarnotacurso():
    n = int(input("ingrese la cantidad de cursos que desea registrar: "))
    for i in range(n):
        nombre = input(f"ingrese el nombre del curso {i + 1}: ").strip()
        if not nombre:
            print("el nombre del curso no puede estar

```

Esta función permite al usuario registrar una cantidad determinada de cursos junto con sus respectivas notas. El flujo incluye validaciones básicas para evitar entradas vacías o fuera de rango.

	<pre> vacío.") return try: nota = float(input("ingrese la nota obtenida (O-100): ")) if nota < 0 or nota > 100: print("la nota debe estar entre 0 y 100.") return except ValueError: print("la nota debe ser un numero.") return historialcursos[nombre] = nota print(f"curso {nombre} registrado correctamente con nota {nota}") </pre>	
<i>mostrar cursos</i>	<pre> def mostrarcursos(historialcursos): if not historialcursos: print("no hay cursos registrados.") else: print("listado de cursos registrados:") for nombre, nota in historialcursos.items(): print(f"{nombre} -{nota: {nota}}") </pre>	Esta función muestra en pantalla todos los cursos registrados junto con sus respectivas notas. Es una función de salida que recorre el contenido del diccionario historialcursos.
<i>promedio general</i>	<pre> def promediogeneral(historialcursos): if not historialcursos: print("no hay cursos registrados para calcular el promedio.") return total = sum(historialcursos.values()) promedio = total / len(historialcursos) print(f"promedio general calculado: {promedio:.2f}") </pre>	Esta función calcula el promedio general de las notas registradas en el diccionario historialcursos.
<i>cursos aprobados cursos reprobados</i>	<pre> def reproapro(historialcursos): aprobados = 0 reprobados = 0 for nota in historialcursos.values(): if nota >= 60: aprobados += 1 else: reprobados += 1 print(f"total de cursos aprobados: {aprobados}") print(f"total de cursos reprobados: {reprobados}") </pre>	Esta función cuenta cuántos cursos han sido aprobados (nota ≥ 60) y cuántos reprobados (nota < 60), y muestra los totales.
<i>buscar curso por nombre, forma lineal</i>	<pre> def busquedalineal func(historialcursos, busquedalineal, historialgeneral): nombre = input("ingrese el nombre del curso a buscar: ").strip() if not nombre: </pre>	Esta función realiza una búsqueda lineal de un curso por nombre en el diccionario historialcursos, registra cuántas veces se ha consultado cada curso y guarda un historial de

	<pre> print("este campo no puede estar vacío") return encontrado = False for curso in historialcursos: if curso.lower() == nombre.lower(): print(f"curso encontrado: {curso} // nota: {historialcursos[curso]}") busquedalineal[curso] = busquedalineal.get(curso, 0) + 1 print(f"este curso ha sido consultado {busquedalineal[curso]} veces.") historialgeneral.append(f"Se busco el curso {curso} - total de búsquedas: {busquedalineal[curso]}") encontrado = True break if not encontrado: print("curso no encontrado.") historialgeneral.append(f"búsqueda fallida: curso {nombre} no encontrado.") </pre>	búsquedas exitosas o fallidas. historialcursos: se recorre con un for para buscar el curso por nombre. busquedalineal: se actualiza con <code>.get(curso, 0) + 1</code> para contar búsquedas. historialgeneral: se usa como una bitácora de texto, agregando mensajes con <code>.append()</code>. Se implementa explícitamente al recorrer el diccionario con un for y comparar cada clave (curso) con el nombre ingresado.
<i>actualizar nombre del curso, nota de curso o ambos</i>	<pre> def editarcurso(nota(historialcursos, historialgeneral): nombre = input("ingrese el nombre del curso que desea actualizar: ").strip() if not nombre: print("este campo no puede estar vacío.") return if nombre not in historialcursos: print(f"el curso '{nombre}' no está registrado.") historialgeneral.append(f"intento fallido de actualización: curso {nombre} no encontrado.") return cursoactual = nombre notaactual = historialcursos[cursoactual] while True: print(f"curso seleccionado: {cursoactual} nota actual: {notaactual}") print("1. editar nombre del curso") print("2. editar nota del curso") print("3. confirmar cambios") try: opcion = int(input("seleccione una opcion: ")) except ValueError: print("ingrese una opción válida.") continue if opcion == 1: nuevonombre = input("ingrese el nuevo nombre del curso: ").strip() if not nuevonombre: print("este campo no puede estar vacío.") continue if nuevonombre in historialcursos: print("ya existe un curso con ese </pre>	Permite al usuario actualizar el nombre o la nota de un curso previamente registrado. También registra los cambios en un historial de acciones. Modificación directa: Se usa <code>.pop()</code> para eliminar una entrada y reasignarla con un nuevo nombre. Actualización de valores: Se modifica la nota directamente con <code>historialcursos[cursoactual] = nuevonota</code>. Registro de eventos: Se usa <code>.append()</code> para guardar mensajes de cambios o intentos fallidos.

	<pre> nombre.") continue historialcursos[nuevonombre] = historialcursos.pop(cursoactual) historialgeneral.append(f"nombre actualizado: '{cursoactual}' -{nuevonombre}") cursoactual = nuevonombre print(f"nombre cambiado a {nuevonombre}.") elif opcion == 2: try: nuevanota = float(input("ingrese la nueva nota (0-100): ")) if nuevanota < 0 or nuevanota > 100: print("la nota debe estar entre 0 y 100.") continue except ValueError: print("la nota debe ser un numero.") continue historialcursos[cursoactual] = nuevanota notaactual = nuevanota historialgeneral.append(f"nota actualizada para {cursoactual}: {nuevanota}") print(f"nota cambiada a {nuevanota}.") elif opcion == 3: print("actualizado... regresando al menu principal...") break else: print("opción no valida.") </pre>	
<i>eliminación de curso</i>	<pre> def eliminarcurso(historialcursos, historialgeneral): if not historialcursos: print("no hay cursos registrados para eliminar.") return nombre = input("ingrese el nombre del curso que desea eliminar: ").strip() if not nombre: print("este campo no puede estar vacio.") return curso_encontrado = None for curso in historialcursos: if curso.lower() == nombre.lower(): curso_encontrado = curso break if curso_encontrado: confirmacion = input(f"seguro que desea eliminar {curso_encontrado} (s/n): ").lower() if confirmacion == 's': historialcursos.pop(curso_encontrado) historialgeneral.append(f"curso eliminado: {curso_encontrado}") print(f"Curso '{curso_encontrado}' eliminado correctamente") </pre>	Permite al usuario eliminar un curso registrado, previa confirmación. También registra el evento (éxito o fallo) en el historial de acciones. Se recorre con un for para buscar el curso por nombre (ignorando mayúsculas/minúsculas). Se elimina con .pop(curso_encontrado) si el usuario confirma Se usa .append() para registrar el resultado de la operación (exitosa o fallida).

	<pre> else: print("la eliminacion fue cancelada") else: print(f"el curso {nombre} no fue encontrado.") historialgeneral.append(f"intento fallido de eliminación: curso {nombre} no existe.") </pre>	
<i>ordenamiento por nota</i>	<pre> def ordenar_por_nota(cursos, historial): if not cursos: print("No hay cursos para ordenar.") return lista = list(cursos.items()) # Convertir el diccionario a lista de tuplas # Ordenamiento burbuja descendente (de mayor a menor) n = len(lista) for i in range(n - 1): for j in range(n - i - 1): if lista[j][1] < lista[j + 1][1]: # Cambiar a "<" para descendente lista[j], lista[j + 1] = lista[j + 1], lista[j] print("\nCursos ordenados por nota (de mayor a menor):") for nombre, nota in lista: print(f"{nombre} - Nota: {nota}") historial.append(f"{nombre} - Nota: {nota}") </pre>	Ordena los cursos registrados en orden descendente según la nota obtenida, utilizando el algoritmo de ordenamiento burbuja. Luego muestra los resultados y los registra en el historial. Se convierte en una lista de tuplas con list(cursos.items()) para poder ordenarlo. lista: se usa para aplicar el algoritmo de ordenamiento burbuja. historial: se actualiza con los resultados ordenados.
<i>ordenamiento por nombre insercion</i>	<pre> def ordenar_alfabetico(cursos, historial): if not cursos: print("No hay cursos para ordenar.") return [] lista = list(cursos.items()) # Convertir el diccionario a lista de tuplas # Algoritmo de inserción (A a Z) for i in range(1, len(lista)): actual = lista[i] j = i - 1 while j >= 0 and lista[j][0].lower() > actual[0].lower(): lista[j + 1] = lista[j] j -= 1 lista[j + 1] = actual print("\nCursos ordenados alfabéticamente (A - Z):") for nombre, nota in lista: print(f"{nombre} - Nota: {nota}") historial.append(f"{nombre} - Nota: {nota}") # Devuelve solo los nombres ordenados return [nombre for nombre, nota in lista] </pre>	Ordena los cursos alfabéticamente por nombre utilizando el algoritmo de ordenamiento por inserción. Luego muestra los resultados y los registra en el historial. Se transforma en una lista de tuplas con list(cursos.items()) para facilitar el ordenamiento. lista: se usa para aplicar el algoritmo de inserción. historial: se actualiza con los resultados ordenados.
<i>buscar nombre de</i>	<pre> def busqueda_binaria(cursos, historial, nombres_ordenados): if not nombres_ordenados: </pre>	Realiza una búsqueda binaria del nombre de un curso en el diccionario cursos, previamente

<i>curso de forma binaria</i>	<pre> print("No hay cursos registrados.") return nombre = input("Ingrese el nombre del curso a buscar (binaria): ").strip() if not nombre: print("El nombre no puede estar vacío.") return izquierda = 0 derecha = len(nombres_ordenados) - 1 encontrado = False while izquierda <= derecha: medio = (izquierda + derecha) // 2 curso_medio = nombres_ordenados[medio] if curso_medio.lower() == nombre.lower(): nota = cursos[curso_medio] print(f"Curso encontrado: {curso_medio} / Nota: {nota}") historial.append(f"Búsqueda binaria exitosa: '{curso_medio}' con nota {nota}") encontrado = True break elif nombre.lower() < curso_medio.lower(): derecha = medio - 1 else: izquierda = medio + 1 if not encontrado: print("Curso no encontrado.") historial.append(f"Búsqueda binaria fallida: curso '{nombre}' no existe.") </pre>	ordenado alfabéticamente. Si lo encuentra, muestra la nota y registra el evento en el historial. Fuente principal de datos. Se accede por clave (curso_medio) para obtener la nota. nombres_ordenados: contiene los nombres de cursos ordenados alfabéticamente. Permite aplicar el algoritmo de búsqueda binaria.
<i>simulación de cola de revisión</i>	<pre> def simular_cola_revision(historialcursos, nom_pers): if not historialcursos: print("No hay cursos en el historial.") return nom2 = input("confirme su nombre para iniciar la revisión: ") if nom2.strip() == "": print("El nombre no puede estar vacío.") return if nom2 != nom_pers: print("El nombre no coincide con el registrado. Revisión cancelada.") return for curso, nota in historialcursos.items(): print(f"Revisando curso: {curso} / Nota: {nota}") print(f"Hola {nom2}, comenzando la revisión de cursos.") print("Simulando cola de revisión...") print("Revisión de todos los cursos completada.") print(f"Gracias por tu paciencia, {nom2}. Revisión finalizada.") print("regresando al menu principal.") </pre>	Simula una revisión secuencial de los cursos registrados, validando primero la identidad del usuario. Representa el concepto de una cola de revisión, donde los cursos se procesan uno por uno. Se recorre con <code>for curso, nota in historialcursos.items()</code> para simular el procesamiento secuencial. Aunque no se usa una estructura <code>queue</code> explícita, el recorrido secuencial de los cursos representa una cola FIFO (First In, First Out). Cada curso se revisa en el orden en que fue registrado en el diccionario (desde Python 3.7, los diccionarios mantienen el orden de inserción).

<i>historial de cambios y búsquedas.</i>	<pre>def mostrarhistorial(historialgeneral): if not historialgeneral: print("no hay historial de acciones.") else: print("historial de acciones realizadas:") for evento in historialgeneral: print(f"- {evento}")</pre>	Muestra en pantalla el historial de acciones realizadas por el usuario durante el uso del sistema, como búsquedas, ediciones, eliminaciones, ordenamientos Se recorre con un for para mostrar cada evento. Se utiliza como bitácora del sistema, permitiendo trazabilidad y auditoría básica.
--	--	---

descripción de cada fase

<i>inicialización</i>	<i>registro de notas</i>
<pre>print("bienvenido al sistema de registro de notas") muestra mensaje de bienvenida nom_pers = input("Ingrese su nombre para empezar: ") para iniciar el programa le pide que ingrese su nombre para avanzar if nom_pers == "": print("El nombre no puede estar vacío. Reinicie el programa.") exit() si deja el campo vacío finaliza el programa print("Nombre registrado, ¡HERE WE GO!") después de ingresar un nombre le muestra este mensaje y luego se le mostrara el menú con todas las opciones disponibles while True: print("Menú principal:") print("1. Agregar curso") print("2. Mostrar cursos") print("3. Calcular promedio") print("4. Conteo de cursos aprobados y reprobados") print("5. Buscar curso por nombre (lineal)") print("6. Actualizar o editar nombre del curso/nota") print("7. Eliminar curso") print("8. Ordenar por nota") print("9. Ordenar por nombre") print("10. Buscar curso por nombre (binaria)") print("11. Simular cola de revisión") print("12. Mostrar historial de cambios") print("13. Salir") es el menú con cada opción disponible enumeradas desde el 1 al 13 try: opcion = int(input("Ingrese el número de la opción: ")) except ValueError: es para pedir un numero dentro del rango 1-13 print("Por favor, ingrese un número válido.")</pre>	<pre>n = int(input("ingrese la cantidad de cursos que desea registrar: ")) al ingresar pide al usuario la cantidad de cursos que desea registrar (n). for i in range(n): Inicia un bucle que va desde 0 hasta n-1 para registrar n cursos. nombre = input(f"ingrese el nombre del curso {i + 1}: ").strip() Pide al usuario el nombre del curso .strip() elimina cualquier espacio antes o después del nombre. if not nombre: print("el nombre del curso no puede estar vacío.") return Verifica si el nombre está vacío (es decir, si no se ingresó nada), Si está vacío, imprime un mensaje de error y termina la función con return. try: nota = float(input("ingrese la nota obtenida (0-100): ")) Pide la nota del curso y la convierte a float. Esto es para permitir notas con decimales if nota < 0 or nota > 100: print("la nota debe estar entre 0 y 100.") return Si la nota no está en el rango entre 0 y 100, muestra un mensaje de error y termina la</pre>

```

    continue
si coloca un numero fuera del rango mostrara este mensaje
    if opcion == 1:
        registrarnotacurso()
    elif opcion == 2:
        mostrarcursos(historialcursos)
    elif opcion == 3:
        promediogeneral(historialcursos)
    elif opcion == 4:
        reproapro(historialcursos)
    elif opcion == 5:
        busquedaalineal_func(historialcursos, busquedaalineal, historialgeneral)
    elif opcion == 6:
        editarcursounota(historialcursos, historialgeneral)
    elif opcion == 7:
        eliminarcurso(historialcursos, historialgeneral)
    elif opcion == 8:
        ordenar_por_notas(cursos=historialcursos, historial=historialgeneral)
    elif opcion == 9:
        ordenar_por_nombre(cursos=historialcursos, historial=historialgeneral)
    elif opcion == 10:
        busqueda_binaria(cursos=historialcursos, historial=historialgeneral)
    elif opcion == 11:
        simular_cola_revision(historialcursos)
    elif opcion == 12:
        mostrarhistorial(historialgeneral)
    break

```

aquí están las opciones disponibles con sus respectivas funciones para hacerlo mas ordenado, cada numero corresponde a una función diferente

```

else:
    print("ingrese una opcion valida")

```

si ingresa algún otro carácter que no sea numero el sistema mostrara este mensaje

función con return.

```

except ValueError:
    print("la nota debe ser un numero.")
    return

```

Si ocurre un error al intentar convertir la entrada a float (es decir, si el usuario ingresa algo que no es un número), captura la excepción ValueError y muestra un mensaje de error.

```

print(f"curso {nombre} registrado correctamente con nota {nota}.")

```

Si todo sale bien, muestra un mensaje indicando que el curso y la nota se registraron correctamente.

mostrar cursos

```
def mostrarcursos(historialcursos):
```

def: Palabra clave para definir una función.
mostrarcursos: Nombre de la función, que indica que su propósito es mostrar los cursos registrados. **historialcursos:** Parámetro que es diccionario donde las claves son nombres de cursos y los valores son las notas.

```

    if not historialcursos:
        print("no hay cursos registrados.")

```

Verifica si el diccionario historialcursos está

promedio general

```
def promediogeneral(historialcursos):
```

Define una función llamada promediogeneral. Recibe un parámetro historialcursos, que se espera sea un diccionario con nombres de cursos como claves y notas como valores.

```

    if not historialcursos:
        print("no hay cursos registrados para calcular el promedio.")
    return

```

Verifica si el diccionario está vacío. **not historialcursos** evalúa a True si no hay cursos registrados. Si no hay cursos, muestra un mensaje y termina la función con

vacío. not historialcursos es una forma de decir: Está vacío o no tiene datos, Si el diccionario está vacío, muestra este mensaje en pantalla. <pre>else: print("listado de cursos registrados:")</pre> Si el diccionario no está vacío, ejecuta el bloque siguiente y muestra un encabezado indicando que se va a listar los cursos. <pre>for nombre, nota in historialcursos.items():</pre> Recorre cada par clave-valor del diccionario. nombre es el nombre del curso (la clave). nota es la nota asociada a ese curso (el valor) .items() devuelve una lista de tuplas con cada curso y su nota. <pre>print(f'{nombre} — nota: {nota}')</pre> Imprime cada curso y su nota en un formato entendible. f'{...}' es una cadena formateada (f-string) que permite insertar variables directamente.	return. <pre>total = sum(historialcursos.values())</pre> historialcursos.values() devuelve una lista de todas las notas. sum(...) suma esos valores. <pre>promedio = total / len(historialcursos)</pre> Calcula el promedio dividiendo el total de notas entre la cantidad de cursos. len(historialcursos) da el número de cursos registrados. <pre>print(f'promedio general calculado: {promedio:.2f}')</pre> Muestra el promedio con dos decimales {promedio:.2f} es una f-string que formatea el número con dos cifras después del punto decimal.
<i>cursos aprobados y reprobados</i> <pre>def reproapro(historialcursos):</pre> Define una función llamada reproapro. Recibe un parámetro historialcursos, que se espera sea un diccionario con nombres de cursos como claves y notas como valores. <pre> aprobados = 0 reprobados = 0</pre> aprobados: cuenta cuántos cursos tienen nota aprobatoria. reprobados: cuenta cuántos cursos tienen nota reprobatoria. <pre> for nota in historialcursos.values():</pre> Recorre todas las notas del diccionario .values() devuelve solo los valores (las notas), ignorando los nombres de los cursos. <pre> if nota >= 61:</pre>	<i>buscar curso de forma lineal</i> <pre>def busqueda_lineal_func(historialcursos, busqueda_lineal, historial_general):</pre> Define la función busqueda_lineal_func. Recibe tres parámetros: historialcursos: diccionario con cursos y sus notas. busqueda_lineal: diccionario que lleva el conteo de búsquedas por curso. historial_general: lista que guarda mensajes de acciones realizadas. <pre> nombre = input("ingrese el nombre del curso a buscar: ").strip()</pre> Solicita al usuario el nombre del curso a buscar .strip() elimina espacios al inicio y al final del texto ingresado <pre> if not nombre: print("este campo no puede estar vacío") return</pre> Verifica si el usuario no ingresó nada. Si el campo está vacío, muestra un mensaje y termina la función. <pre> encontrado = False Variable de control para saber si el curso fue encontrado. for curso in historialcursos:</pre>

```

    aprobados += 1
else:
    reprobados += 1

```

Si la nota es mayor o igual a 61, se considera aprobada y se incrementa el contador aprobados. Si es menor a 61, se incrementa el contador reprobados.

```

print(f"total de cursos aprobados: {aprobados}")
print(f"total de cursos reprobados: {reprobados}")

```

Muestra en pantalla cuántos cursos fueron aprobados y cuántos reprobados. Usa **f-strings** para insertar los valores directamente en el texto.

Recorre cada nombre de curso en el diccionario **historialcursos**.

```

if curso.lower() == nombre.lower():

```

Compara el nombre ingresado con el nombre del curso, ignorando mayúsculas/minúsculas.

```

print(f"curso encontrado: {curso} // nota: {historialcursos[curso]}")

```

Muestra el nombre del curso y su nota.

```

busquedalineal[curso] = busquedalineal.get(curso, 0) + 1

```

Actualiza el contador de búsquedas para ese curso. Si el curso no ha sido buscado antes, lo inicia en 0 y suma 1.

```

print(f"este curso ha sido consultado {busquedalineal[curso]} veces.")

```

Muestra cuántas veces se ha buscado ese curso.

```

historialgeneral.append(f"Se busco el curso {curso} = total de busquedas: {busquedalineal[curso]}")

```

Agrega un mensaje al historial general indicando la búsqueda exitosa y el número de veces que se ha buscado.

```

    encontrado = True

```

```

    break

```

Marca que el curso fue encontrado y termina el bucle.

```

if not encontrado:

```

```

    print("curso no encontrado.")

```

```

    historialgeneral.append(f"busqueda fallida: curso {nombre} no encontrado.")

```

Si no se encontró el curso, muestra un mensaje y lo registra en el historial como búsqueda fallida.

actualizar nombre del curso, nota de curso o ambos

```

def editarcursoynota(historialcursos, historialgeneral):

```

Define la función **editarcursoynota**. Recibe dos parámetros: **historialcursos**: diccionario con cursos y sus notas. **historialgeneral**: lista para registrar los cambios realizados.

```

    nombre = input("ingrese el nombre del curso que desea actualizar: ").strip()

```

Pide al usuario el nombre del curso a editar. **.strip()** elimina espacios al inicio y al final

```

    if not nombre:

```

```

        print("este campo no puede estar vacío.")

```

```

    return

```

Si el usuario no escribe nada, muestra un mensaje y termina la función.

```

    if nombre not in historialcursos:

```

```

        print(f"el curso '{nombre}' no esta registrado.")

```

```

    historialgeneral.append(f"intento fallido de actualizacion: curso {nombre} no encontrado.")

```

```

    return

```

Verifica si el curso existe en el diccionario. Si no existe, informa al usuario y registra el intento fallido en el

eliminar curso

```

def eliminarcurso(historialcursos, historialgeneral):

```

Define la función **eliminarcurso**. Recibe dos parámetros: **historialcursos**: diccionario con cursos y sus notas. **historialgeneral**: lista que registra las acciones realizadas.

```

    if not historialcursos:

```

```

        print("no hay cursos registrados para eliminar:")

```

```

        return

```

Verifica si el diccionario está vacío. Si no hay cursos, muestra un mensaje y termina la función.

```

    nombre = input("ingrese el nombre del curso que desea eliminar: ").strip()

```

Solicita al usuario el nombre del curso que quiere eliminar. **.strip()** elimina espacios al inicio y al final

```

    if not nombre:

```

```

        print("este campo no puede estar vacío.")

```

```

    return

```

Si el usuario no escribe nada, muestra un mensaje y termina la función.

```

    curso_encontrado = None

```

historial.

```
cursoactual = nombre
notaactual = historialcursos[cursoactual]
```

Guarda el nombre y la nota actual del curso para trabajar con ellos durante la edición.

```
while True:
```

Inicia un bucle que permite editar varias veces hasta que el usuario confirme los cambios.

```
    print(f'curso seleccionado: {cursoactual} nota actual: {notaactual}')
    print("1. editar nombre del curso")
```

```
    print("2. editar nota del curso")
    print("3. confirmar cambios")
```

Muestra el curso actual y las opciones disponibles.

```
    try:
        opcion = int(input("seleccione una opcion: "))
    except ValueError:
        print("ingrese una opción valida.")
        continue
```

Solicita una opción numérica. Si el usuario ingresa algo no numérico, muestra un error y vuelve a pedir.

```
    if opcion == 1:
        nuevonombre = input("ingrese el nuevo nombre del curso: ").strip()
        if not nuevonombre:
            print("este campo no puede estar vacio.")
            continue
        if nuevonombre in historialcursos:
            print("ya existe un curso con ese nombre.")
            continue
        historialcursos[nuevonombre] = historialcursos.pop(cursoactual)
        historialgeneral.append(f'nombre actualizado: {cursoactual} {nuevonombre}')
        cursoactual = nuevonombre
        print(f'nombre cambiado a {nuevonombre}.')
```

Pide el nuevo nombre del curso. Verifica que no esté vacío ni duplicado. Actualiza el nombre en el diccionario y en el historial.

```
    elif opcion == 2:
        try:
            nuevanota = float(input("ingrese la nueva nota (0-100): "))
            if nuevanota < 0 or nuevanota > 100:
                print("la nota debe estar entre 0 y 100.")
                continue
        except ValueError:
            print("la nota debe ser un numero.")
            continue
        historialcursos[cursoactual] = nuevanota
        notaactual = nuevanota
        historialgeneral.append(f'nota actualizada para {cursoactual}: {nuevanota}')
        print(f'nota cambiada a {nuevanota}.')
```

Pide una nueva nota. Verifica que sea un número entre 0 y 100. Actualiza la nota en el diccionario y en el historial.

```
    elif opcion == 3:
        print("actualizado.. regresando al menu principal..")
```

Inicializa una variable para guardar el nombre del curso si se encuentra.

```
    for curso in historialcursos:
        if curso.lower() == nombre.lower():
            curso_encontrado = curso
            break
```

Recorre los cursos registrados. Compara el nombre ingresado con cada curso, ignorando mayúsculas/minúsculas. Si lo encuentra, guarda el nombre original y sale del bucle.

```
    if curso_encontrado:
```

Verifica si se encontró el curso

```
    confirmacion = input(f'seguro que desea eliminar {curso_encontrado} (s/n): ').lower()
    if confirmacion == 's':
        historialcursos.pop(curso_encontrado)
        historialgeneral.append(f'curso eliminado: {curso_encontrado}')
    print(f'Curso {curso_encontrado} eliminado correctamente')
```

Si el usuario confirma con "s", elimina el curso del diccionario. Registra la eliminación en el historial. Muestra un mensaje de éxito.

```
    else:
        print("la eliminacion fue cancelada")
```

Si el usuario no confirma, cancela la operación

```
    else:
        print(f'el curso {nombre} no fue encontrado.')
    historialgeneral.append(f'intento fallido de eliminación: curso {nombre} no existe')
```

Si no se encontró el curso, informa al usuario y registra el intento fallido en el historial.

<pre> break Sale del bucle y regresa al menú principal. else: print("opción no valida.") Si el número ingresado no es 1, 2 o 3, muestra un mensaje de error. </pre>	
<i>ordenar en base a la nota</i>	<i>ordenar en base al nombre</i>
<pre> def ordenar_por_nota(cursos, historial): </pre> Define una función llamada ordenar_por_nota que recibe dos parámetros: cursos: un diccionario con pares {nombre_curso: nota}. historial: una lista donde se registrarán los resultados del ordenamiento. <pre> if not cursos: print("No hay cursos para ordenar.") return </pre> Verifica si el diccionario cursos está vacío, Si no hay cursos, muestra un mensaje y sale de la función usando return. <pre> lista = list(cursos.items()) </pre> Convierte el diccionario en una lista de tuplas. <pre> n = len(lista) </pre> Guarda la cantidad de elementos de la lista en n_{SEP}. Se usa para controlar los ciclos del algoritmo burbuja. <pre> for i in range(n - 1): for j in range(n - i - 1): </pre> Estos dos bucles anidados son la base del método burbuja. El primer for controla el número de pasadas. El segundo for compara elementos adyacentes en cada pasada. Con cada iteración, el elemento con la nota más alta "sube" hacia el inicio de la lista. <pre> if lista[j][1] < lista[j + 1][1]: lista[j], lista[j + 1] = lista[j + 1], lista[j] </pre> Aquí ocurre la comparación e intercambio: $lista[j][1]$ es la nota del elemento actual. $lista[j + 1][1]$ es la nota del siguiente elemento. Si la nota actual es menor, se intercambian posiciones$_{\text{SEP}}$. Esto ordena de mayor a menor (descendente). <pre> print("Cursos ordenados por nota (de mayor a menor):") </pre> Muestra un encabezado limpio en pantalla. <pre> for nombre, nota in lista: print(f"{nombre} - Nota: {nota}") historial.append(f"{nombre} - Nota: {nota}") </pre> Recorre la lista ordenada: Imprime el nombre y la nota. Guarda el resultado formateado en el historial (una lista de strings).	<pre> def ordenar_alfabetico(cursos, historial): </pre> Define otra función con los mismos parámetros (cursos y historial) <pre> if not cursos: print("No hay cursos para ordenar.") return </pre> Verifica si el diccionario cursos está vacío, Si no hay cursos, muestra un mensaje y sale de la función usando return. <pre> lista = list(cursos.items()) </pre> Convierte el diccionario en una lista de tuplas. <pre> for i in range(1, len(lista)): actual = lista[i] j = i - 1 </pre> Este es el inicio del método de inserción. Comienza desde el segundo elemento (índice 1). Guarda temporalmente el elemento actual en actual. j sirve para moverse hacia atrás en la parte ya ordenada. <pre> while j >= 0 and lista[j][0].lower() > actual[0].lower(): lista[j + 1] = lista[j] j -= 1 </pre> Mientras no lleguemos al inicio ($j \geq 0$)$_{\text{SEP}}$ y el nombre anterior ($lista[j][0]$) sea alfabéticamente mayor$_{\text{SEP}}$, mueve el elemento hacia adelante ($lista[j + 1] = lista[j]$). <pre> lista[j + 1] = actual </pre> Cuando encuentra la posición correcta, inserta el elemento actual en su lugar. Así se construye una lista ordenada de A a Z. <pre> print("Cursos ordenados alfabéticamente (A - Z):") </pre> Muestra el encabezado del resultado. <pre> for nombre, nota in lista: print(f"{nombre} - Nota: {nota}") historial.append(f"{nombre} - Nota: {nota}") </pre> Imprime y guarda en el historial el resultado del ordenamiento. <pre> return [nombre for nombre, nota in lista] </pre> Devuelve una lista con solo los nombres ordenados, lo cual puede ser útil para operaciones posteriores como la búsqueda binaria.
<i>búsqueda de curso en base al nombre</i>	<i>simulacion</i>

```
def busqueda_binaria(cursos, historial, nombres_ordenados):
```

Define una función llamada **busqueda_binaria** que recibe tres parámetros: **cursos**: el diccionario con los cursos y sus notas. **historial**: la lista donde se registran las acciones realizadas. **nombres_ordenados**: una lista con los nombres de los cursos ya ordenados alfabéticamente.

```
    if not nombres_ordenados:
        print("No hay cursos registrados.")
        return
```

Verifica si la lista de nombres ordenados está vacía. Si no hay cursos disponibles, muestra un mensaje y termina la función.

```
    nombre = input("Ingrese el nombre del curso a buscar (binaria): ").strip()
    if not nombre:
        print("El nombre no puede estar vacío.")
        return
```

Pide al usuario que ingrese el nombre del curso a buscar.

Si el campo está vacío, muestra un aviso y regresa al menú principal.

```
    izquierda = 0
    derecha = len(nombres_ordenados) - 1
    encontrado = False
```

Inicializa tres variables necesarias para la búsqueda binaria: izquierda: el índice inicial de la lista (0). derecha: el índice final de la lista (longitud - 1). encontrado: un indicador lógico que servirá para saber si el curso fue hallado o no.

```
    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2
        curso_medio = nombres_ordenados[medio]
```

Mientras el límite izquierdo no sobrepase al derecho: Calcula el índice del **elemento central** (medio). Obtiene el nombre del curso ubicado en esa posición (curso_medio).

```
    if curso_medio.lower() == nombre.lower():
        nota = cursos[curso_medio]
        print(f"Curso encontrado: {curso_medio} / Nota: {nota}")
        historial.append(f"Búsqueda binaria exitosa: '{curso_medio}' con nota {nota}")
        encontrado = True
        break
```

Compara el curso central con el nombre buscado (ignorando mayúsculas y minúsculas con .lower()). Si son iguales: Recupera la nota del curso. Muestra el resultado al usuario. Registra el evento en el historial. Marca encontrado = True y sale del bucle con break.

```
    elif nombre.lower() < curso_medio.lower():
        derecha = medio - 1
    else:
        izquierda = medio + 1
```

Si el nombre buscado es **alfabéticamente menor** que

```
def simularColaRevisión(historialcursos, nom_pers):
```

Define una función llamada **simularColaRevisión**. Recibe dos parámetros: **historialcursos**: un diccionario con los cursos y sus notas. **nom_pers**: el nombre registrado originalmente por el usuario.

```
    if not historialcursos:
        print("No hay cursos en el historial.")
        return
```

Verifica si el diccionario está vacío. Si no hay cursos registrados, muestra un mensaje y termina la función con return.

```
    nom2 = input("confirme su nombre para iniciar la revisión: ")
```

Solicita al usuario que confirme su nombre antes de iniciar la revisión

```
    if nom2.strip() == "":
        print("El nombre no puede estar vacío.")
        return
```

Elimina espacios en blanco al inicio y al final del nombre ingresado. Si el campo está vacío, muestra un mensaje y termina la función.

```
    if nom2 != nom_pers:
        print("El nombre no coincide con el registrado. Revisión cancelada.")
        return
```

Compara el nombre ingresado (**nom2**) con el nombre original (**nom_pers**). Si no coinciden exactamente, muestra un mensaje y cancela la revisión.

```
    for curso, nota in historialcursos.items():
        print(f"Revisando curso: {curso} / Nota: {nota}")
```

Recorre cada curso en el diccionario. Muestra el nombre del curso y su nota, simulando que está siendo revisado.

```
    print(f"Hola {nom2}, comenzando la revisión de cursos...")
    print("Simulando cola de revisión...")
    print("Revisión de todos los cursos completada.")
    print(f"Gracias por tu paciencia, {nom2}. Revisión finalizada.")
    print("regresando al menu principal...")
```

Muestra mensajes que simulan el proceso de revisión y dan un cierre amigable a la función.

el curso central, el algoritmo busca en la mitad izquierda de la lista (reduciendo el índice derecho). Si es mayor, busca en la mitad derecha (aumentando el índice izquierdo). Así, en cada iteración, la búsqueda se reduce a la mitad del rango anterior. <pre> if not encontrado: print("Curso no encontrado.") historial.append(f"Búsqueda binaria fallida: curso '{nombre}' no existe.") </pre> Si después del bucle while la variable encontrado sigue siendo False, significa que el curso no existe en el registro. En ese caso, se informa al usuario y se agrega el evento al historial.	
<i>historial de cambios y búsqueda</i>	<i>salir</i>
<pre> def mostrarhistorial(historialgeneral): </pre> Define una función llamada mostrarhistorial. Recibe un parámetro historialgeneral, que se espera sea una lista de cadenas de texto (eventos registrados). <pre> if not historialgeneral: print("no hay historial de acciones.") </pre> Verifica si la lista está vacía. Si no hay eventos registrados, muestra un mensaje indicando que no hay historial. <pre> else: print("historial de acciones realizadas:") </pre> Si la lista tiene contenido, muestra un encabezado para la sección del historial. <pre> for evento in historialgeneral: print(f"- {evento}") </pre> Recorre cada elemento de la lista historialgeneral. Muestra cada evento precedido por un guion, en formato de lista.	<pre> elif opcion == 13: print(f"Gracias por usar el sistema, {nom_pers}, adiosito") </pre> con el numero 13 en panel principal será para finalizar el sistema de gestor de notas, eso hace que muestre este mensaje

codigo completo

```

historialcursos = {}
historialgeneral = []
busquedalineal = {}

# 1. Registro de notas de cursos
def registronotacurso():
    n = int(input("ingrese la cantidad de cursos que desea registrar: "))
    for i in range(n):
        nombre = input(f"ingrese el nombre del curso {i + 1}: ").strip()
        if not nombre:
            print("el nombre del curso no puede estar vacío.")
            return
    try:
        nota = float(input("ingrese la nota obtenida (0-100): "))
        if nota < 0 or nota > 100:

```



```

        print("la nota debe estar entre 0 y 100. ")
        return
    except ValueError:
        print("la nota debe ser un numero.")
        return
    historialcursos[nombre] = nota
    print(f"curso {nombre} registrado correctamente con nota {nota}.")

# 2. Mostrar cursos
def mostrarcursos(historialcursos):
    if not historialcursos:
        print("no hay cursos registrados.")
    else:
        print("listado de cursos registrados:")
        for nombre, nota in historialcursos.items():
            print(f"{nombre} -nota: {nota}")

# 3. Promedio general
def promediogeneral(historialcursos):
    if not historialcursos:
        print("no hay cursos registrados para calcular el promedio.")
        return
    total = sum(historialcursos.values())
    promedio = total / len(historialcursos)
    print(f"promedio general calculado: {promedio:.2f}")

# 4. Conteo de cursos aprobados y reprobados
def reproapro(historialcursos):
    aprobados = 0
    reprobados = 0
    for nota in historialcursos.values():
        if nota >= 60:
            aprobados += 1
        else:
            reprobados += 1
    print(f"total de cursos aprobados: {aprobados}")
    print(f"total de cursos reprobados: {reprobados}")

# 5. Búsqueda lineal
def busquedalineal_func(historialcursos, busquedalineal, historialgeneral):
    nombre = input("ingrese el nombre del curso a buscar: ").strip()
    if not nombre:
        print("este campo no puede estar vacio")
        return
    encontrado = False
    for curso in historialcursos:
        if curso.lower() == nombre.lower():
            print(f"curso encontrado: {curso} // nota: {historialcursos[curso]}")
            busquedalineal[curso] = busquedalineal.get(curso, 0) + 1
            print(f"este curso ha sido consultado {busquedalineal[curso]} veces.")
            historialgeneral.append(f"Se busco el curso {curso} - total de busquedas: {busquedalineal[curso]}")
            encontrado = True
            break
    if not encontrado:
        print("curso no encontrado.")
        historialgeneral.append(f"busqueda fallida: curso {nombre} no encontrado.")

# 6. Actualizar o editar nombre del curso/nota
def editarcursonota(historialcursos, historialgeneral):
    nombre = input("ingrese el nombre del curso que desea actualizar: ").strip()
    if not nombre:
        print("este campo no puede estar vacío.")
        return

```

```

if nombre not in historialcursos:
    print(f"el curso '{nombre}' no esta registrado.")
    historialgeneral.append(f"intento fallido de actualizacion: curso {nombre} no encontrado.")
    return
cursoactual = nombre
notaactual = historialcursos[cursoactual]
while True:
    print(f"curso seleccionado: {cursoactual}  nota actual: {notaactual}")
    print("1. editar nombre del curso")
    print("2. editar nota del curso")
    print("3. confirmar cambios")
    try:
        opcion = int(input("seleccione una opcion: "))
    except ValueError:
        print("ingrese una opción valida.")
        continue
    if opcion == 1:
        nuevonombre = input("ingrese el nuevo nombre del curso: ").strip()
        if not nuevonombre:
            print("este campo no puede estar vacio.")
            continue
        if nuevonombre in historialcursos:
            print("ya existe un curso con ese nombre.")
            continue
        historialcursos[nuevonombre] = historialcursos.pop(cursoactual)
        historialgeneral.append(f"nombre actualizado: '{cursoactual}' - '{nuevonombre}'")
        cursoactual = nuevonombre
        print(f"nombre cambiado a {nuevonombre}.")
    elif opcion == 2:
        try:
            nuevanota = float(input("ingrese la nueva nota (0-100): "))
            if nuevanota < 0 or nuevanota > 100:
                print("la nota debe estar entre 0 y 100.")
                continue
        except ValueError:
            print("la nota debe ser un numero.")
            continue
        historialcursos[cursoactual] = nuevanota
        notaactual = nuevanota
        historialgeneral.append(f"nota actualizada para {cursoactual}: {nuevanota}")
        print(f"nota cambiada a {nuevanota}.")
    elif opcion == 3:
        print("actualizado... regresando al menu principal...")
        break
    else:
        print("opción no valida.")
# 7. Eliminar curso
def eliminarcursos(historialcursos, historialgeneral):
    if not historialcursos:
        print("no hay cursos registrados para eliminar.")
        return
    nombre = input("ingrese el nombre del curso que desea eliminar: ").strip()
    if not nombre:
        print("este campo no puede estar vacio.")
        return
    curso_encontrado = None
    for curso in historialcursos:
        if curso.lower() == nombre.lower():

```

```

        curso_encontrado = curso
        break
    if curso_encontrado:
        confirmacion = input(f"seguro que desea eliminar {curso_encontrado} (s/n): ").lower()
        if confirmacion == 's':
            historialcursos.pop(curso_encontrado)
            historialgeneral.append(f"curso eliminado: {curso_encontrado}")
            print(f"Curso '{curso_encontrado}' eliminado correctamente")
        else:
            print("la eliminacion fue cancelada")
    else:
        print(f"el curso {nombre} no fue encontrado.")
        historialgeneral.append(f"intento fallido de eliminaci3n: curso {nombre} no existe.")
# 8. Ordenar por nota (burbuja)
def ordenar_por_nota(cursos, historial):
    if not cursos:
        print("No hay cursos para ordenar.")
        return
    lista = list(cursos.items()) # Convertir el diccionario a lista de tuplas
    # Ordenamiento burbuja descendente (de mayor a menor)
    n = len(lista)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if lista[j][1] < lista[j + 1][1]: # Cambiar a "<" para descendente
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
    print("Cursos ordenados por nota (de mayor a menor):")
    for nombre, nota in lista:
        print(f"{nombre} - Nota: {nota}")
        historial.append(f"{nombre} - Nota: {nota}")
# 9. Ordenar por nombre (inserci3n)
def ordenar_alfabetico(cursos, historial):
    if not cursos:
        print("No hay cursos para ordenar.")
        return []

    lista = list(cursos.items()) # Convertir el diccionario a lista de tuplas

    # Algoritmo de inserci3n (A a Z)
    for i in range(1, len(lista)):
        actual = lista[i]
        j = i - 1
        while j >= 0 and lista[j][0].lower() > actual[0].lower():
            lista[j + 1] = lista[j]
            j -= 1
        lista[j + 1] = actual

    print("Cursos ordenados alfab3ticamente (A - Z):")
    for nombre, nota in lista:
        print(f"{nombre} - Nota: {nota}")
        historial.append(f"{nombre} - Nota: {nota}")

    # Devuelve solo los nombres ordenados
    return [nombre for nombre, nota in lista]
# 10. Búsqueda binaria
def busqueda_binaria(cursos, historial, nombres_ordenados):
    if not nombres_ordenados:
        print("No hay cursos registrados.")
        return

```

```

nombre = input("Ingrese el nombre del curso a buscar (binaria): ").strip()
if not nombre:
    print("El nombre no puede estar vacío.")
    return

izquierda = 0
derecha = len(nombres_ordenados) - 1
encontrado = False

while izquierda <= derecha:
    medio = (izquierda + derecha) // 2
    curso_medio = nombres_ordenados[medio]

    if curso_medio.lower() == nombre.lower():
        nota = cursos[curso_medio]
        print(f"Curso encontrado: {curso_medio} / Nota: {nota}")
        historial.append(f"Búsqueda binaria exitosa: '{curso_medio}' con nota {nota}")
        encontrado = True
        break
    elif nombre.lower() < curso_medio.lower():
        derecha = medio - 1
    else:
        izquierda = medio + 1

if not encontrado:
    print("Curso no encontrado.")
    historial.append(f"Búsqueda binaria fallida: curso '{nombre}' no existe.")

# II. Simular cola de revisión
def simular_cola_revision(historialcursos, nom_pers):
    if not historialcursos:
        print("No hay cursos en el historial.")
        return
    nom2 = input("confirme su nombre para iniciar la revisión: ")
    if nom2.strip() == "":
        print("El nombre no puede estar vacío.")
        return
    if nom2 != nom_pers:
        print("El nombre no coincide con el registrado. Revisión cancelada.")
        return
    for curso, nota in historialcursos.items():
        print(f"Revisando curso: {curso} / Nota: {nota}")
    print(f"Hola {nom2}, comenzando la revisión de cursos...")
    print("Simulando cola de revisión...")
    print("Revisión de todos los cursos completada.")
    print(f"Gracias por tu paciencia, {nom2}. Revisión finalizada.")
    print("regresando al menu principal...")

# 12. Mostrar historial de cambios
def mostrarhistorial(historialgeneral):
    if not historialgeneral:
        print("no hay historial de acciones.")
    else:
        print("historial de acciones realizadas:")
        for evento in historialgeneral:
            print(f"- {evento}")

# Menú principal
print("bienvenido al sistema de registro de notas")

```

```

nom_pers = input("Ingrese su nombre para empezar: ")
if nom_pers == "":
    print("El nombre no puede estar vacío. Reinicie el programa.")
    exit()

print("Nombre registrado, ¡HERE WE GO!")

while True:
    print("Menú principal:")
    print("1. Agregar curso")
    print("2. Mostrar cursos")
    print("3. Calcular promedio")
    print("4. Conteo de cursos aprobados y reprobados")
    print("5. Buscar curso por nombre (lineal)")
    print("6. Actualizar o editar nombre del curso/nota")
    print("7. Eliminar curso")
    print("8. Ordenar por nota")
    print("9. Ordenar por nombre")
    print("10. Buscar curso por nombre (binaria)")
    print("11. Simular cola de revisión")
    print("12. Mostrar historial de cambios")
    print("13. Salir")

    try:
        opcion = int(input("Ingrese el número de la opción: "))
    except ValueError:
        print("Por favor, ingrese un número válido.")
        continue

    if opcion == 1:
        registrar_notas()
    elif opcion == 2:
        mostrar_cursos(historial_cursos)
    elif opcion == 3:
        promedio_general(historial_cursos)
    elif opcion == 4:
        reproapro(historial_cursos)
    elif opcion == 5:
        busqueda_lineal_func(historial_cursos, busqueda_lineal, historial_general)
    elif opcion == 6:
        editar_curso_notas(historial_cursos, historial_general)
    elif opcion == 7:
        eliminar_curso(historial_cursos, historial_general)
    elif opcion == 8:
        ordenar_por_notas(historial_cursos, historial_general)
    elif opcion == 9:
        nombres_ordenados = ordenar_alfabetico(historial_cursos, historial_general)
    elif opcion == 10:
        # La búsqueda binaria requiere la lista ordenada por nombre
        if 'nombres_ordenados' in locals():
            busqueda_binaria(historial_cursos, historial_general, nombres_ordenados)
        else:
            print("Primero debe ordenar los cursos por nombre (opción 9).")
    elif opcion == 11:
        simular_cola_revision(historial_cursos, nom_pers)
    elif opcion == 12:
        mostrar_historial(historial_general)
    elif opcion == 13:

```

```
print("Gracias por usar el sistema, {nom_pers}. ¡Adios! ")
break
else:
    print("Ingrese una opción válida.")
```